PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 1
3. Product Purpose 2
4. System Functionalities 2
5. User Stories & Use Cases 6
6. Features Included (Scope Definition) 7
7. Features Not Included (Scope Control) 7
8. Assumptions & Constraints 7
9. Risks & Questions Throughout Development 10

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Stratatouille - AI Spending Decision Assistant

Version: 2.0

1. Project Overview

- **Problem Statement:** Students often struggle with "impulse buying" and fail to visualize the long-term consequences of their daily spending. Traditional budgeting apps only track past data. They do not provide real-time, proactive advice on whether a user should make a purchase based on their specific financial context.
- **Target Domain:** Enabling rapid UI/UX for prototypes and early stages of the solution-based product development.
- **Proposed Solution Summary:** A prototype of a web application that acts as an AI partner. It uses a student's financial situation to help them go from "blind spenders" to "informed decision-makers".

2. Background & Business Objective

- **Background of the Problem:** Most financial tracking is manual and linear. A user spends, then records it, and eventually realizes they are out of money. This reactive process lacks of immediate guidance at the point of purchase.
- **Importance of Solving This Issue:** By accelerating financial analysis from "end-of-month review" to "instant simulation", students can avoid debt and build better habits. It breaks the barrier of needing complex spreadsheets to understand one's financial health.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Strategic Fit / Impact:** Aligns with the move toward AI-Native personal assistants. The app integrates the Z AI ILMU-GLM-5.1 with a Flutter front-end and Firebase backend to create a seamless Google ecosystem workspace.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-enabled partner that helps students validate spending decisions.

3.2. Intended Users (Target Audience):

- University students
- Young adults with limited allowances

4. System Functionalities

4.1. Description: The system operates as a generative engine for financial logic. It captures relationships between daily spending, monthly allowances, and future financial stability.

4.2. Key Functionalities:

- **Dashboard:** Display the user's financial status (e.g., status, balances, spending, categories).
- **Add Expense:** Enables users to add expenses, including the items, amount and category.
- **AI Chat:** Provide a live AI spending assistant interaction for users to ask a questions regarding the financial.
- **Affordability Checker:** Provide an instant feedback to users whether they can afford to buy an item or not.

© UMHackathon 2026

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Z AI ILMU-GLM-5.1. It offers high-speed inference and is OpenAI-compatible, making it ideal for real-time AI chat and rapid “Can I afford this” feedback.

4.3.2. **Prompting Strategy: Zero-shot prompting** with structured output enforcement.

The system uses zero-shot prompting. No example input-output pairs are provided in the prompt. Instead, the prompt gives the model a role description, the user's financial context, and a strict output format to follow.

Why zero-shot fits this use case:

- **Domain is narrow and well-defined.** The model only needs to reason about budget arithmetic (subtraction, comparison) and category matching. These are tasks that modern LLMs handle reliably without examples — few-shot would add token cost without meaningful accuracy gains.
- **Structured output format acts as a soft constraint.** By demanding Answer/Reason/BasedOn, the prompt channels the model into producing parseable, explainable responses. This replaces the need for a few-shot examples that would demonstrate the format.
- **Cost efficiency.** Zero-shot prompts are the shortest possible prompts. Given the three-tier fallback architecture (LLM → deterministic → keyword), the LLM is already a "nice to have" rather than a necessity. Adding a few-shot examples would increase input tokens by 3–5x per call for marginal quality improvement.
- **The fallback system compensates for zero-shot's weakness.** Zero-shot prompting can produce format deviations, but the app doesn't rely on perfect LLM output — ExplainabilityService and the keyword-based fallback catch and repair malformed responses. This makes zero-shot's lower reliability acceptable.

© UMHackathon 2026

4.3.3. **Context & Input Handling**:

The system applies **input trimming at multiple layers**, but there is **no explicit token counting, chunking, or rejection mechanism.** There is **no hard limit on user input size**. A user could paste a 10,000-character message into the chat field, and it would be forwarded as-is to the backend, which would forward it as-is to the LLM API. The only practical limit is:

- The ilmu-glm-5.1 model's own context window (the API will return an error if the input exceeds the model's maximum context length).
- The **12-second timeout** on the backend (main.py:59) and **14-second timeout** on the Flutter client (glm_service.dart:79) — oversized inputs may cause the request to time out before the model finishes processing.

What happens to inputs that exceed limits:

1. **If the model API rejects the input** (context window exceeded): The backend raises an exception, the try/except catches it, and the keyword-based deterministic fallback activates (main.py:201-301). The user sees a generic budget-based answer tagged as [Fallback].
2. **If the request times out**: The http.post(...).timeout() throws a TimeoutException, same fallback path.
3. **If the input is valid but very long**: The model processes it, but the response may be delayed or the model may lose focus on the core question, producing off-topic or incomplete output — handled by the fallback mechanisms in 4.3.4.

4.3.4. **Fallback & Failure Behavior:**

The system has a **three-tier fallback architecture** with no retry mechanism and no human escalation path:

Tier 1: LLM response (source: `"glm"`)

If the model returns a valid, parseable response with all three sections (`Answer`, `Reason`, `BasedOn`), it is displayed directly with an "AI response" label.

Tier 2: Deterministic fallback (source: `"deterministic"` or `"glm_failed"`)

When the LLM call fails, times out, or returns malformed output, the backend switches to rule-based logic:

- **"Recent spending" queries** → formatted list of the 4 most recent expenses
- **Affordability queries** → arithmetic comparison against remaining budget with tiered advice (≤30% = "Yes", ≤100% = "Be careful", >100% = "No")
- **Category prediction** → keyword matching against common terms ("lunch" → Food, "grab" → Transport, etc.)
- **Reduce/overspend queries** → top spending category identification

These responses are labeled "AI fallback response" in the UI.

Tier 3: Client-side fallback (source: `"glm_failed"`)

If the backend itself is unreachable, the Flutter client catches the exception and displays: *"Unable to generate a full AI response right now."* Reason: "The system could not load current data or contact the AI service."

What's missing:

- **No retry mechanism** — a failed LLM call is not retried. The system immediately falls back. Given the 12-second timeout, retrying would double the user's wait time, so this is an intentional trade-off.
- **No human escalation path** — there is no "contact support", "report issue", or "ask a human" option. The user can only rephrase and try again.
- **No hallucination detection** — the `BasedOn` field is presented as a data reference, but its contents are never cross-checked against the actual budget/expense numbers. The user

© UMHackathon 2026
Email: umhackathon@um.edu.my

has no way to distinguish a grounded AI response from a fabricated one beyond the `[AI]` vs `[Fallback]` label.

- **No content filtering** — offensive, harmful, or irrelevant LLM outputs are not filtered before display.

5. User Stories & Use Cases

- **User Stories:** As a student, I want to ask the AI if I can afford an item, for instance, an RM50 concert ticket, so that I don't run out of money for food before the end of the month. I also want to ask whether my current financial status is good, whether I am overspending or not.

- **Use Cases (Main Interactions):**

Prompt to UI Flow: Users enter a natural-language question, then the system parses the intent, assembles financial context, routes to the appropriate response path, and renders a structured explainability card

Flow Generation: The app uses a shared Firestore stream as the single source of truth, with **`BudgetService`** as the shared computation layer that maintains consistent financial logic across all screens.

Data continuity model:

Every screen reads from the same Firestore stream independently — there is no passed state between screens.

How the four screens connect:

1. **Dashboard**: read-only. Displays metrics computed by `BudgetService` (risk level, projected balance, category breakdown, alerts). No AI calls. Changes in Firestore automatically refresh the UI via `StreamBuilder`.

© UMHackathon 2026

2. **Add Expense**: write-only (to Firestore). User enters amount, note, category (with AI auto-categorize option). Once saved to Firestore, the Dashboard's stream automatically picks up the new expense on the next frame.
3. **AI Chat**: read + AI. Fetches current budget state on each message send, assembles context, calls the backend. The AI's response is not persisted — it exists only in the local `_messages` list for that session. Starting a new session clears all chat history.
4. **Affordability Checker**: read + AI. Same pattern as chat: fetches fresh budget data per check, calls the backend, displays result. Result is held in local state only.

6. Features Included (Scope Definition)

- AI Chat: A conversational workspace to ask AI regarding financial questions.
- Affordability Checker: An instant feedback from AI regarding the "Can I afford to buy this?" question.
- Firestore Sync: Real-time synchronization of expenses across the team.

7. Features Not Included (Scope Control)

- Direct Bank Integration: The app does not link to real bank accounts. It relies on manual user input.
- Production Environment: This version is a functional prototype for the UMHackathon.

8. Assumptions & Constraints

- **LLM Cost Constraint:**

  Token Usage Estimates: The projected financial overhead per typical user engagement is approximately RM 0.01 to RM 0.03, reflecting a consumption rate of 500 to 1,500 tokens per session.

  Design Decision for Cost Efficiency: Managing inference overhead via a tiered fallback architecture with deterministic outputs. The primary strategy for controlling operational

© UMHackathon 2026

expenses involves a **three-tier response architecture**. This system avoids unnecessary LLM processing by identifying common query patterns:

- **GLM (LLM) Path:** Executes model inference only for complex or novel user queries.
- **Deterministic Path:** Utilizes direct list formatting for "recent spending" requests, bypassing the AI engine entirely.
- **Keyword Fallback:** Employs rule-based logic for affordability checks and spending advice if the model times out or fails.

Furthermore, the chat interface restricts contextual data to **only four recent expenses**. This ensures a compact and predictable token count per session, regardless of the user's total transaction history.

- **Technical Constraints:**
  - **Platform & Output Constraint:** Development is strictly restricted to the Flutter framework (Android/iOS/Web/macOS). The system does not utilize native Swift or Kotlin modules, nor does it support code generation or the exportation of financial reports in PDF, CSV, or Excel formats.
  - **Backend Environment:** The backend architecture utilizes Python (FastAPI) running on a local host at 127.0.0.1:8000. It lacks cloud deployment, horizontal scaling, and load balancing; Android emulators must interface via 10.0.2.2.
  - **Currency Constraint:** Financial data is locked to the Malaysian Ringgit (RM). The current prototype does not feature multi-currency support.
  - **Data & Connectivity:** Persistence is managed exclusively through Cloud Firestore. The application lacks local SQLite caching, necessitating a persistent internet connection for all functional operations.
  - **AI Pathway Architecture:** The system identifies two distinct pathways: AiService (direct Anthropic API) and GLMService (backend proxy). Active implementation is limited to the backend proxy, while the direct API in ai_service.dart remains a legacy component.

© UMHackathon 2026

- **Performance Constraints:**
  - **LLM Latency & Timeout Policy:** The backend environment enforces a 12-second timeout on GLM calls, while the Flutter client operates with a 14-second threshold. In instances of non-responsiveness, the system initiates the predefined fallback sequence.
  - **Token Generation Capacity:** Output is strictly capped at 1,024 tokens per request. This technical limitation necessitates concise AI responses and may impact the depth of explanations for complex, multi-dimensional user queries.
  - **Non-Streaming Interface:** The application does not support data streaming; responses are delivered only upon full generation completion. Consequently, users will view a loading indicator throughout the entire processing duration.
  - **Stateless Response Architecture:** The system lacks response caching or memoization mechanisms. Identical or repeated questions trigger unique LLM calls, resulting in duplicate token consumption and operational costs for recurrent queries.
  - **UI Text Constraints:** AI-generated response text is automatically truncated at 300 characters within the chat interface. This UI constraint effectively mandates high levels of brevity and conciseness from the model.
  - **Contextual Budget Limitation:** Financial analysis and budget simulations are restricted to a single-month window. The current month's data is utilized exclusively, while historical transaction records remain outside the active context window.

- **User Input:**
  - **Proactive Interaction Constraint:** The system is restricted from generating unsolicited financial advice. Every AI engagement necessitates an explicit user-initiated trigger, such as a manual query or the activation of the affordability check mechanism, ensuring no autonomous insights or notifications are produced.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Manual Expense Categorization Approval:** While the system utilizes the ILMU-GLM-5.1 to predict transaction categories, these suggestions are not auto-applied. Users maintain complete oversight and must manually validate or override the AI prediction via the dropdown interface before data persistence occurs.
- **Affordability Checker Input Requirements:** Evaluations require structured data comprising both a specific item descriptor and a positive numerical amount. The engine is unable to process ambiguous or open-ended inquiries that lack a defined Malaysian Ringgit (RM) valuation for the prospective purchase.
- **Guided Query Architecture:** The chat interface employs suggestion chips to direct users toward optimized query structures. This design ensures that interactions remain within the functional scope of the AI logic, as free-form or unstructured questions may trigger generic fallback responses.
- **Prototype Environment Constraints:** This version remains a functional prototype for the UMHackathon and lacks production-grade features. Technical limitations include local host dependency, manual API key management via environment files, and the absence of robust rate limiting or user authentication protocols.

9. Risks & Questions Throughout Development

- **Design Similarity:**

  Risk of Response Uniformity: The system may generate nearly identical financial advice for users with similar budget ratios due to the static nature of prompt templates. For instance, two distinct users with RM 500 remaining inquiring about an RM 50 expense might receive redundant feedback, failing to account for their unique long-term spending behaviors.

  Current Mitigation Measures (Limited Scope):

© UMHackathon 2026

- The prompt architecture dynamically injects the user's **actual recent transaction history** into the context to encourage variance in AI responses based on historical data.
- The affordability engine utilizes a *feature_type* flag (differentiating between 'chat', 'affordability', or 'categorize') to adjust system instructions accordingly.

Remaining Personalization Gaps:

- **Contextual Data Bottleneck:** Personalisation is restricted by the **four-expense limit** within the chat interface. This prevents the AI from distinguishing between vastly different user spending profiles if their most recent transactions appear similar.
- **Absence of User Personas:** The prompt design lacks demographic signals or spending personas, treating students and high-income professionals with identical financial logic.
- **Generic Fallback Logic:** Deterministic paths rely on rule-based keyword matching, resulting in repetitive advice such as "Reduce [top category]" for all users sharing a primary expense type.
- **Lack of Deduplication:** The system lacks response caching, increasing operational overhead as it fails to recognize or suppress duplicate AI advice across different user sessions.

- **Frame Stability:**

  Structural Format Risk: The AI model may generate responses that deviate from the mandatory *Answer/Reason/BasedOn* schema. This inconsistency can lead to the interface displaying unformatted text or empty content blocks.

  Current Mitigation Measures:

  - **Enforced Schema Validation:** The *ExplainabilityService.formatResponse()* method mandates the three-tier structure for all fallback outputs.

© UMHackathon 2026

- **GLM Integrity Check:** The *GLMService.ensureExplainabilityFormat()* function verifies the presence of required labels (answer, reason, and based on) within the LLM output, automatically reformatting raw text if segments are missing.
- **Tolerant Extraction Logic:** The system utilizes regex-based parsing in *ExplainabilityService.parseResponseSections()* to accommodate variations in spacing and casing.
- **Graceful UI Degradation:** In scenarios where specific fields return null, the chat interface reverts to displaying the raw string to maintain visibility.

Remaining Risks (Not Fully Mitigated):

- **Brittle Parser Dependency:** The backend relies on a rigid line-by-line parser that expects exact label matches. Complex markdown formatting or merged lines from the LLM could result in data corruption or missing sections.
- **UI Overflow Vulnerability:** There is a lack of character limits for the answer and basedOn fields in the UI. While reasons are truncated, extremely long AI responses may compromise the integrity of the card layout.
- **Lack of Retry Logic:** The application currently lacks a mechanism to re-trigger LLM calls upon receiving malformed responses, forcing users to view broken or generic fallback content.
- **Detection String Conflict:** Source labels for [AI] and [Fallback] are appended as raw text. If the AI model generates the string "[AI]" within its response, the system may incorrectly identify the data source.

- **Drawing Interpretation**:

  Risk of Contextual Ambiguity: Users may engage the system with vague or off-domain inquiries (e.g., "help" or "money stuff") that fall outside the current AI contextual window. The system's limited natural language understanding (NLU) necessitates a heavy reliance on rule-based keyword matching for fallback routing.

  Current Mitigation Measures:

© UMHackathon 2026

- **Keyword-Based Routing Architecture:** The backend logic employs pattern detection to trigger specific functional paths: "afford" initiates affordability checks, while terms like "reduce" or "save" generate optimization advice.
- **Regex Value Extraction:** The system utilizes regular expressions to isolate Malaysian Ringgit (RM) values from free-text inputs, ensuring numerical data is correctly parsed for simulation.
- **Deterministic Categorization Fallback:** In instances of LLM failure, the backend reverts to a rule-based matching engine to predict transaction categories based on common merchant keywords.
- **Structured Input Validation:** The affordability checker enforces mandatory data requirements—requiring an item descriptor and a positive numeric amount—prior to initiating model inference.

Remaining Risks (Not Fully Mitigated):

- **Absence of Out-of-Domain Rejection:** The system lacks a "no-intent" state. Off-topic queries (e.g., weather or crypto) may result in misleading, generic budget advice rather than a graceful error response.
- **Multi-Intent Parsing Limitations:** Complex queries involving multiple financial intents are processed as a single intent; the engine identifies the initial keyword and disregards subsequent user requirements.
- **Free-Text Disambiguation Gap:** While the dedicated checker screen validates input, the chat interface does not require specific amounts, potentially leading to hallucinated values or unhelpful AI feedback.
- **Vernacular & Colloquial Incompatibility:** The rule-based fallback exclusively targets English terminology, failing to accommodate Malaysian users utilizing code-switched or colloquial Malay terms (e.g., "makan").
- **Suggestion Chip Bias:** The user interface encourages pre-defined query structures, making interactions that deviate from these scripts more susceptible to hitting unhandled edge cases.

© UMHackathon 2026

Email: umhackathon@um.edu.my